Quest 5: The Cartograffity

Lord of the Wings · growing land, sand and trees on a grid

> This guide uses the official puzzle text. The story is summarised and the rules are restated in plain words. Every number in the examples and traces was checked by running the code in this repo (`node partN.ts --sample`, plus small instrumented copies to print intermediate values).

In one sentence: start from the centre cell of a 0/1 grid, grow land outwards with a neighbour-count rule, and count cells of one kind.
Input: a grid of `0` (light) and `1` (dark) with an odd number of rows and columns.
Output: a count of cells: land (Part I), sand (Part II) or trees (Part III).
What changes between parts: each part keeps everything from the previous one and adds a layer: land → a sandy coastline on the land → trees on the land that is left.

1. The story

Legolink has joined the expedition, and the Grand Council hands over maps to Nulldor. The maps look like static: just light and dark patches. Legolink teaches everyone to read them: from the centre, the land "grows" over the dark patches, then a beach forms around its edge, and finally trees get stamped onto the solid ground with a potato stamp.

2. Understanding the question

The grid and the eight neighbours

Every rule in this quest looks at a cell's **eight neighbours**: up, down, left, right and the four diagonals. Rows and columns are counted from 0, so the sample (11 rows × 21 columns) has its centre at **row 5, column 10**.

```
NW  N  NE
 W  X  E        X has 8 neighbours (fewer on the edge of the map,
SW  S  SE       only 3 in a corner)
```

Rule 1: growing land (all parts)

1. The centre cell is land `#`.
2. A cell that **touches the land** (including diagonally) becomes land if at least **5** of its 8 neighbours are "good". A good neighbour is land `#` or a dark cell `1`.
3. Repeat until nothing changes.

A small example of my own: the middle cell `?` below touches land (top right). Its good neighbours are `1`, `#`, `1`, `1`, `1`, which makes 5, so it becomes land. If one of those `1`s were a `0`, it would have only 4 and would stay water.

```
1 0 #
1 ? 0
1 1 0
```

Note that "touches the land" matters. A blob of `1`s far away from the centre never becomes land, however dark it is.

Rule 2: sand (Parts II and III)

1. Every land cell with at least one **water** neighbour (a `0` or `1` that didn't become land) becomes sand `=`. The map counts as surrounded by water, so land on the map's edge becomes sand too.
2. Then, repeatedly, a land cell with at least **5** sand neighbours also becomes sand.

Rule 3: trees (Part III)

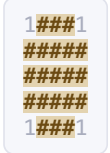
The stamp is a 5×5 shape with its corners cut off. Every `+` must be land `#` or a tree `@`:

```
. + + + .
+ + + + +
+ + X + +      20 cells marked +
+ + + + +
. + + + .
```

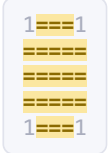
Land cells are tried in reading order (top to bottom, left to right), and a new tree counts straight away for later cells. After stamping, the forest grows **twice**: a land cell becomes a tree if it has at least **4** tree neighbours and **no** sand neighbour. These two steps are **simultaneous**: a tree added in a step only counts in the next step.

Extra examples of my own (all run through the real code)

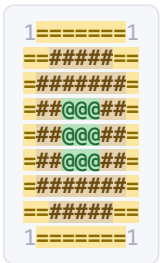
Map	Land (I)	Sand (II)	Trees (III)	Why it's interesting
5×5 all `0`	1	1	0	Nothing can grow. The single centre cell touches water, so it becomes sand.
5×5 all `1`	21	21	0	The 4 corners never become land: a corner has only 3 neighbours, so it can't reach 5. Everything else ends up sand.
9×9 all `1`	77	32	9	A thick enough island: a 3×3 block of trees in the middle.
11×11 all `1`	117	40	25	A 5×5 block of trees in the middle.



5×5 all 1: land



5×5 all 1: sand



9×9 all 1: final

Shared code: reading the grid (identical in all three parts)

```
3  const isSample = process.argv.includes("--sample");
4  const file = isSample ? "sample1.txt" : "input1.txt";
5  const INPUTS: string[][] = readFileSync(file, "utf8")
6    .trim()
7    .split("\n")
8    .map((i) => i.split(""));
9
10 const totalRows = INPUTS.length;
11 const totalCols = INPUTS[0]?.length || 0;
12
13 INPUTS[Math.floor(totalRows / 2)]![Math.floor(totalCols / 2)] = "#";
```

q5/part1.ts, lines 3–13

Lines	What happens
3–4	`--sample` picks `sample1.txt`; otherwise the real input for that part.
5–8	Reads the file, drops the trailing newline, splits it into rows, and splits each row into single characters. So `INPUTS[row][col]` is `"0"` or `"1"`. Using an array of arrays (not strings) means cells can be changed in place.
10–11	Grid size: 11 rows × 21 columns for the sample.
13	`Math.floor(11 / 2) = 5` and `Math.floor(21 / 2) = 10`: the centre cell becomes `#`.

> **Fragile, not wrong:** line 7 splits on `\n` only. If an input file were ever saved with Windows line endings (`\r\n`), every row except the last would end in a `\r` character. Then `totalCols` would be 22 instead of 21, and the "centre" would be in the wrong column. Your input files are fine, so this doesn't affect your answers.

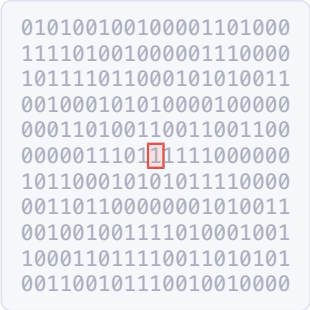
Lines 1–47 (reading the grid and growing the land) are the same in all three parts. Parts II and III paste them in and add more code below.

3. Part I: how much land?

The question

Grow the land from the centre (Rule 1) until it stops, then count the `#` cells.

Worked example (sample)



the notes, centre outlined



after growing: 137 land cells

The puzzle draws the growth in **rounds**: in every round, all cells that qualify are added together. A round-by-round copy of your code shows that the sample needs 10 rounds after the centre, adding 4, 8, 17, 20, 22, 21, 20, 13, 8, 3 cells. That's 1 + 136 = **137**, matching the puzzle's 11 diagrams.

Step-by-step solution

1. Read the grid into a 2D array of characters.
2. Mark the centre cell (row $\lfloor \text{rows}/2 \rfloor$, column $\lfloor \text{cols}/2 \rfloor$) as `#`.
3. Sweep over every cell that isn't land yet. Count its land neighbours and its good neighbours (land or `1`).
4. If it has at least 1 land neighbour and at least 5 good ones, make it `#`.
5. Repeat the sweep until one sweep changes nothing.
6. Count the `#` cells.

My solution

```
15  const isValidLand = (row: number, col: number) => {
16    let good = 0;
17    let land = 0;
18
19    for (let r = row - 1; r <= row + 1; r++) {
20      if (r < 0 || r >= totalRows) continue;
21
22      for (let c = col - 1; c <= col + 1; c++) {
23        if (c < 0 || c >= totalCols || (r === row && c === col)) continue;
24        if (INPUTS[r]![c] === "#") land++;
25        if (INPUTS[r]![c] === "1" || INPUTS[r]![c] === "#") good++;
26      }
27    }
28
29    return land >= 1 && good >= 5;
30  };
31
32  let changed = 1;
33
34  while (changed != 0) {
35    changed = 0;
36
37    for (let row = 0; row < totalRows; row++) {
38      for (let col = 0; col < totalCols; col++) {
39        if (INPUTS[row]![col] === "#") continue;
40
41        if (isValidLand(row, col)) {
42          INPUTS[row]![col] = "#";
43          changed++;
44        }
45      }
46    }
47  }
48
49  let sum = 0;
50
51  for (let row = 0; row < totalRows; row++) {
52    for (let col = 0; col < totalCols; col++) {
53      if (INPUTS[row]![col] === "#") sum++;
54    }
55  }
56
57  console.log(sum);
```

q5/part1.ts, lines 15–57

Lines	What happens
15–17	`isValidLand` counts two things around `(row, col)`: `land` (how many `#`) and `good` (how many `#` or `1`).
19–23	Visits the 3×3 block around the cell. Rows or columns outside the map are skipped (`continue`), and so is the cell itself (`r === row && c === col`).
24–25	A `#` neighbour counts towards both totals. A `1` counts only as good.
29	`land >= 1` is the "touches the land" rule. `good >= 5` is the 5-of-8 rule.
32–35	Keeps sweeping the whole grid while the last sweep changed something.
39	Cells that are already land are skipped.
41–44	If the cell qualifies, it becomes `#` **immediately**, and the rest of this sweep already sees it.
49–57	Counts the `#` cells and prints the total.

Trace: sweeps of the `while` loop (sample)

Sweep	Cells added (changed)	Land after the sweep
1	51	52
2	24	76
3	24	100
4	19	119
5	9	128
6	7	135
7	2	137
8	0	137 → loop stops

> **Why in-place still gives the puzzle's answer:** your code needs 7 working sweeps, but the puzzle needs 10 rounds. That's because a cell added early in a sweep can help cells later in the same sweep: sweep 1 already adds 51 cells, while the puzzle's first round adds only 4. The final map is the same anyway, because the rule only ever gets *easier* to satisfy. A cell becoming land turns a neighbour from "not good" into "good" (or keeps it good), and it never removes land. So the order in which cells are added can't change which cells are added in the end. This is a **fixed point** of a **monotone** rule.

> **The map's edge:** lines 20 and 23 treat cells outside the map as not good. Because of this, a corner (3 neighbours) can never become land, and an edge cell needs all 5 of its neighbours to be good. That's how the 5×5 all-`1` example ends up with 21 land cells, not 25.

4. Part II: how much sand?

The question

Grow the land as in Part I. Then turn the coastline into sand: first every land cell that touches water (or the map's edge), then repeatedly every land cell with at least 5 sand neighbours. Count the `=` cells.

Worked example (sample)

```
01=100100100001101000
1========0000====0000
1==#####==00==##=0011
00==#####====###==000
000=############=100
000=############==000
101=############=0000
00==############=0011
00=#############==001
10======#=========101
0011001===10010010000
```

coastline: 59 sand

```
01=100100100001101000
1========0000====0000
1====###==00=====0011
00==#####====##===000
000=############==100
000=############==000
101=############=0000
00==############=0011
00===##########===001
10================101
0011001===10010010000
```

after growing: 69 sand

The puzzle shows the sand growing in 3 rounds. A round-by-round copy of your code adds 6, 3 and 1 cells: 59 + 10 = **69**.

Step-by-step solution

1. Do Part I to get the land.
2. Go over every land cell once. If any cell in its 3×3 block is off the map, or is `0`/`1`, make it `=`.
3. Sweep over the remaining land cells. If one has at least 5 `=` neighbours, make it `=`.
4. Repeat until one sweep changes nothing.
5. Count the `=` cells.

My solution

Lines 1–47 are Part I unchanged (apart from the input file name on line 4).

```
const isTouchesWater = (row: number, col: number) => {
  for (let r = row - 1; r <= row + 1; r++) {
    if (r < 0 || r >= totalRows) return true;
    for (let c = col - 1; c <= col + 1; c++) {
      if (c < 0 || c >= totalCols) return true;
      if (INPUTS[r]![c] === "0" || INPUTS[r]![c] === "1") return true;
    }
  }

  return false;
};

for (let row = 0; row < totalRows; row++) {
  for (let col = 0; col < totalCols; col++) {
    if (INPUTS[row]![col] !== "#") continue;

    if (isTouchesWater(row, col)) INPUTS[row]![col] = "=";
  }
}

const isValidSand = (row: number, col: number) => {
  let sand = 0;

  for (let r = row - 1; r <= row + 1; r++) {
    if (r < 0 || r >= totalRows) continue;

    for (let c = col - 1; c <= col + 1; c++) {
      if (c < 0 || c >= totalCols || (r === row && c === col)) continue;
      if (INPUTS[r]![c] === "=") sand++;
    }
  }

  return sand >= 5;
};

changed = 1;

while (changed != 0) {
  changed = 0;

  for (let row = 0; row < totalRows; row++) {
    for (let col = 0; col < totalCols; col++) {
      if (INPUTS[row]![col] !== "#") continue;

      if (isValidSand(row, col)) {
        INPUTS[row]![col] = "=";
        changed++;
      }
    }
  }
}

let sum = 0;

for (let row = 0; row < totalRows; row++) {
  for (let col = 0; col < totalCols; col++) {
    if (INPUTS[row]![col] === "=") sum++;
  }
```

```
}

console.log(sum);
```

q5/part2.ts, lines 49–109 (highlighted lines are the ones that changed from the previous part)

Lines	What happens
49–59	`isTouchesWater` returns `true` as soon as the 3×3 block around the cell goes off the map (lines 51, 53) or contains a `0` or `1`. The cell itself is in the block too, but it is `#`, so it never matches.
61–67	One pass over the grid: every `#` that touches water becomes `=`.
54	This only counts `0`/`1` as water, **not** "anything that isn't `#`". That matters, because cells earlier in this same pass have already become `=`. If `=` counted as water, the sand would spread inward through the whole island in one pass.
69–82	`isValidSand` counts `=` neighbours (off-map cells are skipped) and asks for at least 5.
84–99	The same "sweep until nothing changes" loop as Part I, but now it turns `#` into `=`.
105	Counts `=` instead of `#`.

Trace (sample)

Stage	Changed	Sand	Land left
after lines 61–67 (coastline)	59	59	78
sand sweep 1	8	67	70
sand sweep 2	2	69	68
sand sweep 3	0	69 → loop stops	68

As in Part I, the in-place sweeps (8 + 2) and the puzzle's rounds (6 + 3 + 1) add the same 10 cells, because the sand rule is also monotone: more sand never stops a cell from qualifying.

5. Part III: how many trees?

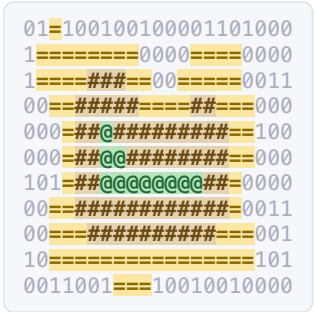
The question

Build land and sand as before. Then stamp trees onto land cells in reading order, using the 20-cell stamp, and let the forest grow for exactly two simultaneous steps (at least 4 tree neighbours, no sand neighbour). Count the `@` cells.

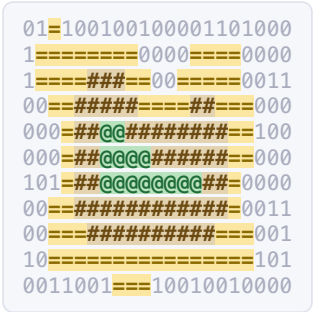
Worked example (sample)



land + sand



after stamping: 11 trees



after 2 growth steps: 14 trees

Stage	Trees added at (row, col)	Total trees
stamping	(4,6) (5,6) (5,7) (6,6) (6,7) (6,8) (6,9) (6,10) (6,11) (6,12) (6,13)	11
growth step 1	(5,8)	12
growth step 2	(4,7) (5,9)	14

The puzzle stamps one cell at a time and says earlier trees can help later positions. In practice the order can't matter: a `+` cell passes if it is `#` *or* `@`, and the stamp only ever turns a `#` into an `@`, so no other cell's check ever changes. Stamping all at once would give the same 11 trees.

Step-by-step solution

1. Do Part II to get the land and sand.
2. Go over land cells in reading order. If all 20 stamp cells around it are on the map and are `#` or `@`, make it `@` right away.
3. Growth step: collect every land cell that has no `=` neighbour and at least 4 `@` neighbours. Then turn all of them into `@` together.
4. Do the growth step a second time.
5. Count the `@` cells.

My solution

Lines 1–99 are Part II unchanged (apart from the input file name on line 4).

```
const isValidTree = (row: number, col: number) => {
  let tree = 0;

  for (let r = row - 2; r <= row + 2; r++) {
    if (r < 0 || r >= totalRows) continue;

    for (let c = col - 2; c <= col + 2; c++) {
      if (c < 0 || c >= totalCols || (r === row && c === col)) continue;
      if ((r === row - 2 || r === row + 2) && (c === col - 2 || c === col + 2))
        continue;
      if (INPUTS[r]![c] === "#" || INPUTS[r]![c] === "@") tree++;
    }
  }

  return tree === 20;
};

for (let row = 0; row < totalRows; row++) {
  for (let col = 0; col < totalCols; col++) {
    if (INPUTS[row]![col] !== "#") continue;
    if (isValidTree(row, col)) INPUTS[row]![col] = "@";
  }
}

const canGrowTree = (row: number, col: number) => {
  let tree = 0;

  for (let r = row - 1; r <= row + 1; r++) {
    if (r < 0 || r >= totalRows) continue;

    for (let c = col - 1; c <= col + 1; c++) {
      if (c < 0 || c >= totalCols || (r === row && c === col)) continue;
      if (INPUTS[r]![c] === "=") return false;
      if (INPUTS[r]![c] === "@") tree++;
    }
  }

  return tree >= 4;
};

for (let i = 0; i < 2; i++) {
  const positions: { row: number; col: number }[] = [];

  for (let row = 0; row < totalRows; row++) {
    for (let col = 0; col < totalCols; col++) {
      if (INPUTS[row]![col] !== "#") continue;
      if (canGrowTree(row, col)) positions.push({ row, col });
    }
  }

  positions.forEach(({ row, col }) => (INPUTS[row]![col] = "@"));
}

let sum = 0;

for (let row = 0; row < totalRows; row++) {
  for (let col = 0; col < totalCols; col++) {
    if (INPUTS[row]![col] === "@") sum++;
```

```
  }
}

console.log(sum);
```

q5/part3.ts, lines 101–162 (highlighted lines are the ones that changed from the previous part)

Lines	What happens
101–116	`isValidTree` checks the 5×5 block around the cell. It skips cells off the map, the cell itself, and the 4 corners (line 109). It counts how many of the rest are `#` or `@`.
115	All 20 `+` cells must pass. Off-map cells are skipped rather than counted, so a cell too close to the edge can never reach 20.
118–123	Tries every `#` in reading order and changes it to `@` **in place**, as the puzzle describes. (As explained above, the order doesn't affect the result here.)
125–139	`canGrowTree`: if any neighbour is `=`, it returns `false` straight away. Otherwise it counts `@` neighbours and needs at least 4.
141–152	Two growth steps. In each one, the qualifying cells are first **collected** in `positions` (line 147) and only changed afterwards (line 151). That makes the step simultaneous.
156–162	Counts `@`.

Why growth must be simultaneous (your code gets it right): the stamp is in place (lines 118–123), but growth is collect-then-apply (lines 141–152). As an experiment, I made a copy where growth also changes cells immediately. On the sample, step 1 then adds (5,8), (5,9), (5,10), (5,11), (5,12): each new tree gives the next cell its 4th tree neighbour. The final count is **17** instead of 14. Unlike Parts I and II, this growth runs for a fixed number of steps rather than until nothing changes, so the order of updates does change the answer.

The map's edge again: `canGrowTree` skips off-map neighbours, so it doesn't see the "water around the map" there. That's safe: any land cell on the edge already became sand in Part II, so the only `#` cells left are inside the map.

6. Summary

Part	Counts	New rule	How cells are updated	Sample
I	land `#`	touches land and ≥ 5 neighbours are `#` / `1`	in place, repeat until no change	137
II	sand `=`	touches water/edge; then ≥ 5 `=` neighbours	coastline in one pass; then in place until no change	69
III	trees `@`	20-cell stamp; then ≥ 4 `@` and no `=` neighbours	stamp in place; growth simultaneous, exactly 2 steps	14

Cost: each sweep looks at every cell and its 8 neighbours (24 for the stamp), so it costs O(rows × cols). The land and sand loops repeat that until nothing changes (8 and 3 sweeps on the sample), which is fast enough for these maps.